



POLITECNICO
MILANO 1863

**SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE**

Network Overview

Mesos - Software Engineering Final Project:
GC06 - Ricciardi, Salomoni, Sciorelli, Sironi

Teacher: Prof. Gianpaolo Cugola
Year 2025-2026

Contents

Contents	II
Nomenclature	III
1 Introduction & High-Level Architecture	V
1.1 Architectural Style	V
1.2 Integration with MVC	V
1.3 Dual Protocol Requirement	V
2 Network Abstraction Layer	VI
2.1 Server-Side Abstraction	VI
2.2 Client-Side Abstraction	VI
3 Communication Protocols Implementation	VII
3.1 TCP Socket Protocol	VII
3.2 RMI (Remote Method Invocation)	IX

	III
4 Server Infrastructure & Matchmaking	X
4.1 Server Bootstrap	X
4.2 Lobby & MatchManager	X
4.3 The Match Component	X
5 Messaging Protocol & Data Routing	XI
5.1 Data Transfer Objects (DTO)	XI
5.2 Double Dispatch & Visitor Pattern	XI
5.2.1 Design Alternatives Considered	XI
6 Concurrency Management: ActionQueue & UI Synchronization	XII
6.1 The ActionQueue Mechanism	XIII

Nomenclature

<i>DTO</i>	Data Transfer Object
<i>RMI</i>	Remote Method Invocation
<i>TCP</i>	Transmission Control Protocol

1 | Introduction & High-Level Architecture

1.1. Architectural Style

The system architecture is based on the Client-Server model, which divides tasks between resource requesters (clients) and resource providers (the server). Clients request services and submit actions, while a centralized server processes transactions, manages shared resources, and broadcasts responses to the connected nodes.

1.2. Integration with MVC

The traditional Model-View-Controller pattern has been distributed across the network:

- The Model and Controller components are exclusively located on the **server**. The server acts as the authoritative controller and holds the only validated, “real” state of the game. It is the sole entity allowed to enforce game rules, validate moves, and mutate the game’s state.
- The clients, host the view and a stripped-down version of the model (named `SmallModel`). The client does not perform any rule checking. It simply takes the user input, forwards it to the server, and updates its local state only when the server notifies a change in the model.

1.3. Dual Protocol Requirement

A core project requirement was to build a network stack capable of natively supporting two radically different communication technologies: TCP Sockets and RMI. This called for a very rigorous abstraction logic in order to avoid duplicating the code

2 | Network Abstraction Layer

2.1. Server-Side Abstraction

The `GameController` and the game state are strictly decoupled from the underlying network protocols. The Server interacts with connected players exclusively through a generic proxy interface named `VirtualClient`.

When an event is resolved, the controller invokes

```
1 virtualClient.notifyPlayer(dto);
```

 Java

At Runtime, dynamic method dispatch seamlessly routes the execution to the appropriate protocol implementation

- if the client is connected via TCP, the `TCPClientManager` implementation serializes the DTO over a `ObjectOutputStream`
- if the client is connected via RMI, the `RMIClientManager` invokes the remote method exposed by the client's stub.

This approach rigorously adheres to the Dependency Inversion Principle, ensuring that the core business logic never depends on low-level I/O details.

2.2. Client-Side Abstraction

Similarly, both the TUI and GUI interact solely with a generic `ServerConnection` interface.

Whenever a user triggers an action, or sends a command, the view dispatches the method ignoring the underlying communication protocol

```
1 client.getServerConnection().placeTotem(...);
```

 Java

It is the concrete implementation's role, chosen in the login phase, to appropriately dispatch the method into a call for a remote method or transform the command in a Socket packet.

The View remains entirely network-agnostic, achieving a complete separation between the graphical interface and the network infrastructure.

3 | Communication Protocols Implementation

3.1. TCP Socket Protocol

- **Asynchronous Full-Duplex Communication:** Communication via socket is engineered to be bi-directional and asynchronous. Both the Server (via `TCPClientReceiver`) and the Client (via `TCPClientConnection`) dedicate specific threads continuously read from their respective `ObjectInputStream` within an infinite `while(true)` loop. This non-blocking architecture ensures that the server can push notifications at any time without freezing the client's UI.
- **Synchronous Fallback via `BlockingBox`:** While asynchrony is highly efficient for gameplay events, certain operations strictly demand an immediate, synchronous response (e.g., login results). In order to overcome Sockets' asynchronous nature, without blocking the rest of the network traffic, a utility called `BlockingBox` was introduced. A `BlockingBox` is a thread-safe object that suspends the calling thread until the listener thread deposits the expected response packet into the "box" with its response, coupling asynchronous requests with their synchronous responses.

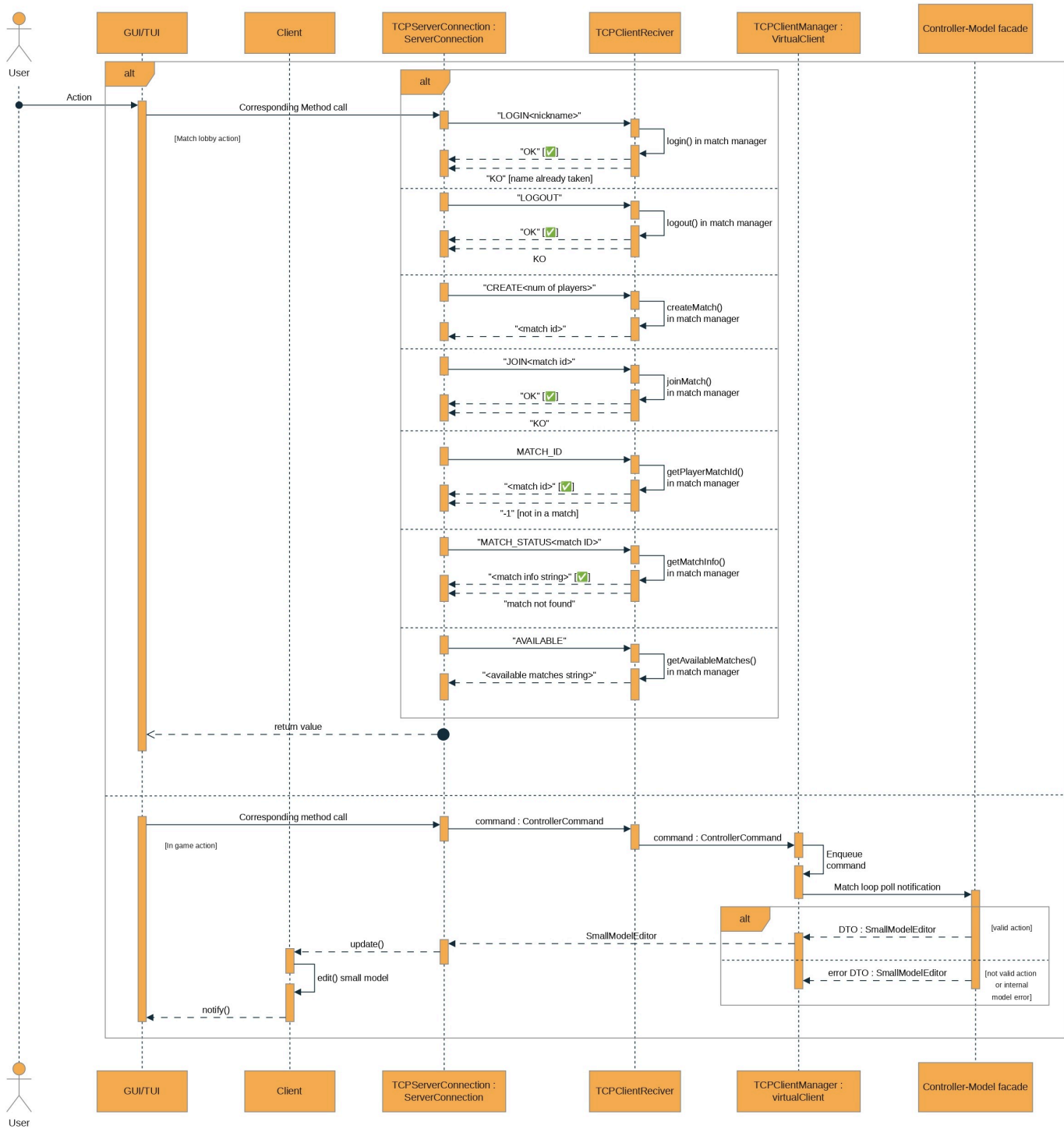


Figure 1: TCP Sequence Diagram

3.2. RMI (Remote Method Invocation)

The native Java RMI implementation masks the network layer, but it is fundamentally synchronous and mono-directional. To allow the server to asynchronously push state updates (DTOs), without the client having to fallback to *polling*, a callback pattern was implemented. The client exposes its own remote interface, passes its stub reference to the server. When the server has to notify a change to the client, it performs a “reverse” remote method invocation on the client’s exposed stub.

This design effectively turns RMI into a bidirectional communication channel that perfectly mirrors the TCP behavior.

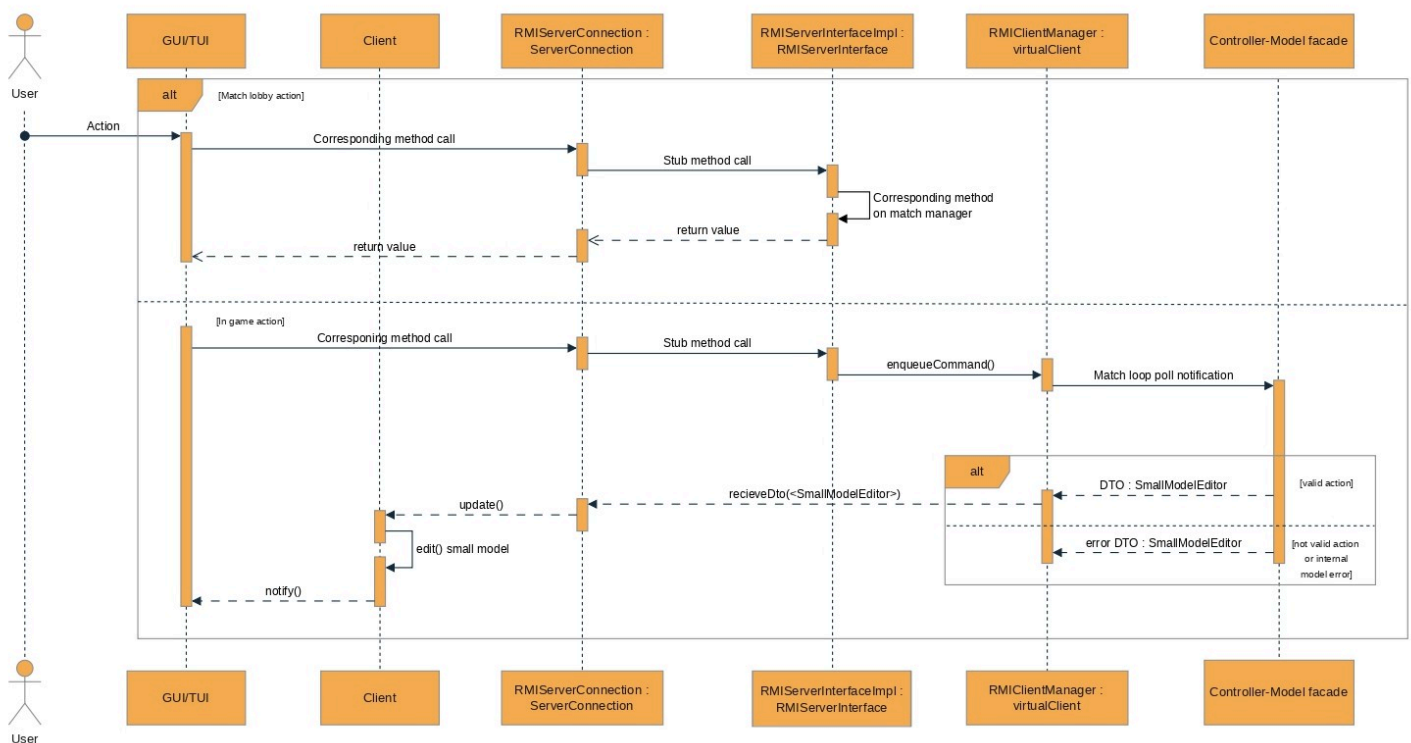


Figure 2: RMI Sequence Diagram

4 | Server Infrastructure & Matchmaking

4.1. Server Bootstrap

The class `ServerMain` acts as the entry point. It is responsible for initializing both the `TCPServer`, on the designated port, as well as RMI Registry on the same host machine, converging all incoming network traffic into a single, unified matchmaking backend.

4.2. Lobby & MatchManager

Upon establishing a connection, clients undergo a validation phase and are left into a logical “waiting room”. The `MatchManager` is in charge of monitoring this state. Because connection handling is an inherently multi-threaded process—with multiple clients potentially connecting and disconnecting simultaneously, the `MatchManager` employs strict thread-safety mechanisms. Critical sections (such as assigning players to a lobby or evaluating the quorum) are rigorously protected via synchronization locks to prevent race conditions and ensure atomic state transitions

4.3. The Match Component

Once a lobby reaches its predefined quorum (selected when creating the match), the `MatchManager` moves the players from the lobby and injects them inside a `Match`. acts as an isolated sandbox, possessing its own dedicated `GameController` and game model, allowing the server to horizontally scale without the risk of state crossover or thread interference

5 | Messaging Protocol & Data Routing

In order to minimize the network load, the server never broadcasts the complete game state. Instead, it transmits lightweight **DTOs** containing all the information that has changed (a “delta”).

5.1. Data Transfer Objects (DTO)

To maintain strict decoupling, these DTOs implement a common abstraction: the `SmallModelEditor` interface.

As such, when a DTO is sent, it is not required by the client to edit the small model itself. Instead, the DTO exposes a function called `edit` that changes the Small Model on the client.

5.2. Double Dispatch & Visitor Pattern

In order to avoid the use of explicit type-checking (`instanceof`), the architecture makes extensive use of the visitor Pattern. The `SmallModelEditor` interface forces the implementation of the `accept(DTOVisitor v)` method. When a client receives a generic DTO message, it instantiates a concrete visitor and invokes:

```
1 dto.accept(visitor)
```

Java

The JVM, through double dispatch, correctly routes the visit method to the right DTO.

This design ensures that introducing a new network event into the game only requires creating a new DTO and adding its corresponding visit method to the Visitor. The core network listening loop remains entirely untouched, perfectly adhering to the Open/Closed Principle.

5.2.1. Design Alternatives Considered

During the network design phase, we deliberated extensively on how to structure the data payloads. Ultimately, we adopted a Hybrid Protocol Architecture, dividing the communication into two distinct phases to balance simplicity and type-safety:

- **Phase 1: The Text-Based Handshake (Lobby & Authentication)** For the initial connection phase—such as logging in, fetching available matches, and joining a lobby—the TCP implementation relies on a lightweight, text-based string protocol (e.g., “LOGIN[nickname]”, “CREATE[players]”). Rationale: At this stage, the client is not yet authenticated or assigned to a specific Match. Instantiating, serializing, and de-serializing complex Java Objects just to check if a nickname is available would introduce unnecessary overhead. Simple string parsing with `startsWith()` provides a fast, synchronous mechanism to handle the handshake.
- **Phase 2: The Object-Based Application Protocol (In-Game)** Once the player is successfully injected into a Match, the communication strictly transitions to a Type-Safe, Object-Oriented Protocol. We explicitly rejected the use of “Magic Strings” (e.g., sending “TOTEM_MOVED” or “PICK_CARD”) for gameplay actions. Instead:
 - **Client-to-Server:** Actions are wrapped in `ControllerCommand` objects utilizing a strictly defined Enum (the Request enum). This guarantees compile-time safety and utilizes the Command Pattern.
 - **Server-to-Client:** The server broadcasts state changes using our polymorphic `SmallModelEditor DTOs`. Rationale: Using native Java Object Serialization for the heavy game state ensures absolute parity between the TCP and RMI implementations. It completely eradicates the risk of typos in action strings.

6 | Concurrency Management: ActionQueue & UI Synchronization

Integrating asynchronous network events with sequential game logic and interactive graphical interfaces introduces severe thread-safety challenges. To guarantee consistency and prevent exceptions, the architecture implements rigorous synchronization mechanisms on both the server and the client.

Because multiple clients can submit actions simultaneously, allowing network threads to modify the game state directly would lead to fatal race conditions.

6.1. The ActionQueue Mechanism

To elegantly solve this synchronization issue, we implemented an ActionQueue system, effectively applying the Producer-Consumer architectural pattern.

- The Producers (Clients): Instead of applying network requests directly to the model, the server's network receiver threads push the incoming requests into a **BlockingQueue**. The network thread simply “drops” the payload in the queue and immediately returns to listening to the socket, maximizing network responsiveness.
- The Consumer (Server): Each Match instance runs on its own dedicated thread. The Match's `matchLoop()` acts as the sole consumer. It continuously polls the **ActionQueue**, popping and executing the **ControllerCommands** strictly sequentially. This guarantees that game state mutations happen atomically. After resolving the logic, the `matchLoop()` broadcasts the resulting DTOs back to the clients.

A similar decoupling strategy is applied on the client to safely render incoming server events. Network DTOs arrive unpredictably on background threads, but User Interfaces must be updated exclusively by their dedicated main threads.